

Reviewer Pack — Minimal Order of Quandle Group Representations vs Monotone C...

Entry 8210fbb91ec8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 20:17:42 UTC

Minimal Order of Quandle Group Representations vs Monotone Circuit Depth

Entry ID: 8210fbb91ec8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 20:17:42 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Quandles) **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity

Statement:

{‘text’: ‘For every instance of the Monotone XOR Circuit problem with n inputs, there exists a quandle group representation of size $O(n^2)$ such that the minimal order of any non-trivial element in the group corresponds to the depth of the monotone circuit.’, ‘quantitative’: ‘ $E[\text{min_order}(\text{quandle_group})] = \Theta(n^2)$ ’}

Rationale (proposer’s reasoning):

{‘text’: ‘Quandles, which are algebraic structures generalizing groups, provide a way to encode boolean functions. The minimal order of elements in a quandle can be related to the complexity of computing certain properties of the associated boolean function. Monotone circuits are known to capture computational complexity for problems like monotone XOR, and by studying the relationship between quandles and these circuits, we may uncover new insights into computation.’}

Taxonomy category: QUANDLE_GROUP_REPRESENTATIONS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0489a7cc6cf53b5f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all $n \leq 40$, the average minimal order of non-trivial elements in quandle group representations is within $O(n^2)$ with a standard deviation $\leq 10\%$ of the mean, and there is a significant positive correlation ($p < 0.05$) between the minimal order and circuit depth.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - quandle group representation AND monotone circuit complexity - algebraic combinatorics quandles AND circuit depth $O(n^2)$ - Monotone XOR Circuit problem AND quandle group minimal order

Top relevant hits considered: - [<http://arxiv.org/abs/2012.15478v3>] N-quandles of links - [<http://arxiv.org/abs/hep-ph/0610012v1>] Tevatron-for-LHC Report of the QCD Working Group - [<http://arxiv.org/abs/0905.3374v3>] Symmetric Extensions of Dihedral Quandles and Triple Points of Non-orientable Surfaces - [<http://arxiv.org/abs/2304.04810v2>] Chain algebras of finite distributive lattices - [<http://arxiv.org/abs/1711.02729v2>] On f- and h- vectors of relative simplicial complexes - [<http://arxiv.org/abs/2102.07269v1>] The combinatorics of Jeff Remmel - [<http://arxiv.org/abs/2303.14697v2>] The central tree property and algorithmic problems on subgroups of free groups - [<http://arxiv.org/abs/2602.24220v1>] Comparing Classical and Quantum Variational Classifiers on the XOR Problem

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=-9, elapsed=179.8s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_invertible(matrix):
        n = len(matrix)
        if n == 0 or not all(len(row) == n for row in matrix):
            return False

    def determinant(A):
        if len(A) == 1:
            return A[0][0]
        det = Fraction(0)
        sign = 1
        for i in range(len(A)):
            submatrix = [row[:i] + row[i+1:] for row in A[1:]]
            det += sign * A[0][i] * determinant(submatrix)
```

```

        sign *= -1
    return det

    return determinant(matrix) != 0

def generate_quandle_group(n):
    quandle_group = []
    for i in range(2**n):
        row = [0] * n
        for j in range(n):
            if (i >> j) & 1:
                row[j] = 1
        quandle_group.append(row)
    return quandle_group

def circuit_depth(circuit):
    depth = 0
    for gate in circuit:
        if gate[0] == 'XOR':
            depth += max(circuit_depth(gate[1]), circuit_depth(gate[2]))
        else:
            depth += 1
    return depth

n = random.randint(5, 40)
quandle_group = generate_quandle_group(n)

min_order = float('inf')
for i in range(len(quandle_group)):
    if is_invertible(quandle_group[i]):
        order = sum(quandle_group[i])
        if order < min_order:
            min_order = order

circuit = []
for _ in range(2**n):
    gate = random.choice(['XOR', 'NOT'])
    if gate == 'XOR':
        inputs = [random.randint(0, 1) for _ in range(n)]
        circuit.append((gate, inputs[:n//2], inputs[n//2:]))
    else:
        input_ = random.randint(0, 1)
        circuit.append((gate, input_, None))

depth = circuit_depth(circuit)

return {
    "metric_name": "Minimal Order of Quandle Group",
    "metric_value": min_order,
    "instances_tested": len(quandle_group),
    "conjecture_holds": min_order <= n**2 and min_order >= 0.9 * n**2,
    "counterexample": "" if conjecture_holds else f"n={n}, min_order={min_order}"
}

if __name__ == "__main__":
    import sys

```

```

seeds = [int(s) for s in sys.argv[1:]] or list(range(30, 150, 2))

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='n={results[0]['instances_tested']}', min_order={min_order}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support conditions could not be evaluated. | next: Re-run the test to collect data and evaluate the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14920
2	preregistration	ollama_remote	glm4:latest	0	0	6313
3	novelty	ollama_remote	glm4:latest	0	0	4590
4	novelty	ollama_remote	glm4:latest	0	0	6934
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40577
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12787

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11960
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10758
9	judge	ollama_remote	glm4:latest	0	0	65511

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 174349 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/8210fbb91ec8.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/8210fbb91ec8.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/8210fbb91ec8.tar.gz (if generated)